

How the Sales Pipeline Works

A plain-language walkthrough of the Customer Sales Analytics pipeline you rebuilt on Databricks Free Edition — what each piece does, why it exists, and how they connect.

The one-sentence version

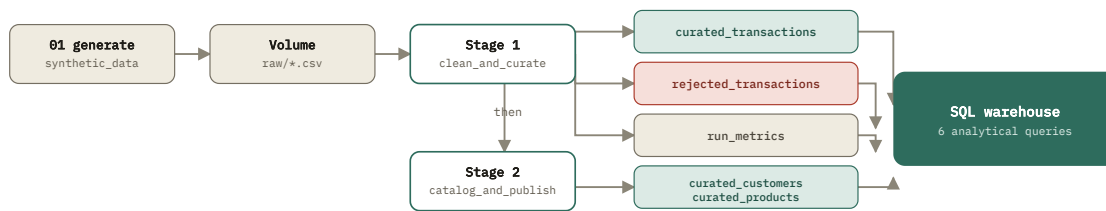
Messy sales data goes in one end; a set of automated steps checks it, fixes what it can, throws out what it can't, and stacks the good rows into tidy tables you can ask questions of — and the whole thing can be re-run safely, any time, without creating duplicates.

THE KITCHEN ANALOGY

Think of it like a restaurant kitchen. Deliveries arrive messy and mixed (**raw**). A prep cook checks every item, tosses what's spoiled or mislabeled into a separate bin (**rejected**), and lines up everything good, chopped and organized, ready to cook (**curated**). A second person then plates it and puts it on the pass so servers can actually bring it to tables (**published / queryable**). That's the entire pipeline — three stations, two workers, one order of operations.

The architecture, end to end

Everything lives inside one Databricks workspace. No external servers, no separate database — just a Volume (plain file storage), a handful of Delta tables (smart, trustworthy tables), two notebooks, and a Workflow that runs them in order.



Two notebooks, chained by a Databricks Workflow – everything downstream reads from Delta tables, never from the raw files directly.

RAW ZONE

A Unity Catalog **Volume** – basically a shared folder Databricks manages for you. Holds the untouched CSVs, deliberately messy on purpose (that's covered below).

REJECTED ZONE

A Delta table holding every row that failed a check, tagged with *why* it failed. Nothing is silently dropped – every reject is explainable.

CURATED ZONE

Delta tables holding only the rows that passed every check – deduplicated, typed correctly, ready for analysis.

Walking through the pipeline, stage by stage

0

01_generate_synthetic_data

Make some deliberately messy data

Real sales data is never clean, so instead of hunting for a real dataset, a small script *invents* 5,000 fake transactions and then deliberately breaks some of them: duplicate order IDs, blank customer fields, negative or nonsense amounts, dates that don't exist (like month 13), and orders that reference a customer or product that was never created.

Why break your own data on purpose? Because a pipeline that's never seen a bad row hasn't proven anything. This gives every cleaning rule downstream something real to catch.

Standardize, then check everything

Before any rule can run, every column needs to actually mean what it says. This step: lowercases and cleans up column names, strips currency symbols so "\$1,204.50" becomes a real number, and parses dates that arrive in two different formats (2025-01-15 or 01/15/2025).

Then four checks run on every row, each producing a plain yes/no answer:

CHECK	ASKS
missing_required_field	Is the order ID, customer, or product blank?
invalid_amount	Is the amount zero, negative, or a wild outlier?
invalid_date	Is the date missing, malformed, or outside a sane range?
orphan_reference	Does the customer or product actually exist?

A row that fails any check gets stamped with the reason it failed and routed toward the rejected pile — first failure wins, so every rejected row has exactly one clear reason.

Remove duplicates, the same way every time

Because the generator injects exact-duplicate order IDs (imagine the same order accidentally uploaded twice), the pipeline needs a rule for which copy survives — and it has to pick the *same* survivor every single time it runs, or two runs on identical input would disagree with each other.

The rule: keep whichever copy was ingested most recently; if that's somehow tied, break the tie by filename. Deterministic, boring, and exactly what you want from a data pipeline.

File everything away, safely re-runnable

Surviving rows get sorted into folders by **year, month, day** of the transaction date — this is called **partitioning**. It matters because almost every question you'd ask this data ("what was March's revenue?") filters by date, and partitioning lets the warehouse skip straight to the relevant slice instead of scanning everything.

WHY "RERUN-SAFE" MATTERS

If you accidentally run this notebook twice on the same day's data, a naive pipeline would double every row. This one uses a Delta Lake feature called **dynamic partition overwrite**: a write only replaces the exact date-folders it touched, so running it five times on the same input leaves the table looking exactly like running it once.

Three tables come out of this stage: `curated_transactions` (the good rows), `rejected_transactions` (the bad rows, with reasons), and `run_metrics` (one line per run recording how many rows went in, how many were rejected, and how many made it through — your audit trail).

Put it on the pass

The second, smaller stage republishes the two reference tables — `curated_customers` and `curated_products` — in full each time (they're small, so there's no need to be clever), and then tidies up the transactions table's file layout so queries against it run fast.

This stage exists mainly to give the pipeline a genuine *second* step — proof that data moves through more than one orchestrated stage before it's considered "ready," same as a real production pipeline would have an ingestion stage and a separate publishing stage.

Tying the two stages together

A **Databricks Workflow** is the thing that actually runs stage 1, waits for it to succeed, and only then runs stage 2 — like a relay race where the second runner physically cannot start until the baton (a successful run) is handed off. Without a Workflow, you'd have two independent notebooks and no proof they run in a meaningful order.

Asking the data questions

Once `curated_transactions` exists, six standing questions get answered with plain SQL against it — no new pipeline needed, just queries against the tables the pipeline already built:

- **Monthly revenue trend** — how much came in, month by month
- **Top 10 products by revenue** — what's actually selling
- **Top 20 customers by lifetime spend** — who your best customers are
- **Average order value by category** — which product categories drive bigger baskets
- **Month-over-month growth** — is revenue accelerating or slowing
- **Reject-rate breakdown** — a data-quality report card, straight from the rejected table

One example, so it's concrete — this is the whole "top products" query, readable almost like English:

```
-- Which products bring in the most money?
SELECT p.product_name, SUM(t.amount) AS revenue
FROM curated_transactions t
JOIN curated_products p ON t.product_id = p.product_id
GROUP BY p.product_name
ORDER BY revenue DESC
LIMIT 10;
```

Where this differs from the original AWS version

Same pipeline, same rules — just different plumbing underneath.

AWS CONSOLE VERSION	DATABRICKS VERSION	WHY IT'S SIMPLER HERE
S3 buckets (raw/rejected/curated folders)	Volume + Delta tables	One managed store instead of a bucket you provision and secure yourself
Glue Crawler	Spark's built-in schema inference	No separate service to configure
Two Glue jobs	Two notebooks	Write and debug interactively, cell by cell
Athena + <code>MSCK REPAIR TABLE</code>	Databricks SQL warehouse	Delta tables track their own partitions automatically
Step Functions	Databricks Workflows	Same idea (task graph with dependencies), built into the same UI

Field notes – real snags and what they meant

Worth keeping, since these come up again if you touch the pipeline later.

DATES THAT "LOOK" VALID BUT AREN'T

`"2019-13-40"` has the right shape (YYYY-MM-DD) but month 13 doesn't exist. Databricks runs in strict (ANSI) mode by default, so parsing that raises an error instead of quietly becoming a blank value. Fix: use `try_to_date()` instead of `to_date()` — same idea, but it always hands back a blank instead of erroring on garbage input.

SOME SETTINGS ARE OFF-LIMITS ON SERVERLESS

Free Edition's serverless compute blocks a handful of session-wide Spark settings for isolation reasons. The fix wasn't to work around the restriction — it was realizing the setting was already being applied the safer way, per `write (.option("partitionOverwriteMode", "dynamic") )`, making the blocked global version redundant anyway.

A SHARED FILE WOULDN'T READ RELIABLY

Keeping the cleaning logic in its own file and `import`-ing it into each notebook is normal practice — but a workspace-file sync quirk made that file unreadable at random. The fix: paste the logic directly into each notebook that needs it. Slightly more repetition, zero dependency on a file that might not be there.

Sales Pipeline Field Guide — a plain-language companion to the Databricks Free Edition build of the Customer Sales Analytics pipeline.